WORKING REFERENCE

CRM Sync — UI Component & ID Registry

Canonical naming + delivery model for the storefront UI system (nav, footer, cart, login, search) across the store and crm-sync.dev — **one addressable `crm-` namespace** so every surface, collection, and bundle lines up. Live endpoints marked; planned ones tracked.

Canonical naming + delivery model for the storefront UI system (nav, footer, cart, login, search) across design-sync.myshopify.com → crm-sync.dev. One addressable `crm-` namespace so every surface, collection, and bundle lines up.

Status: working reference. Endpoints marked ✓ are live; ⏳ are planned.

---

1. Delivery model — three app bundles, loaded from the CF Worker

CRM, PIM, and Design each ship as a **separate component bundle** served from the Cloudflare Worker's `/embed/*` endpoints. Updating a bundle = a worker deploy; the storefront never re-pastes code. (Layer 4 of the Higher-Order Stack.)

BUNDLE	LOADS	ENDPOINT (WORKER)	PLACEMENT
Stack (Layer 1)	GA4 (Consent-Mode-v2, id from `/stack/config`) + UIkit/GSAP/petite-vue per-need	`/embed/stack-loader.js` ✓	**helmet** (`<head>`)
CRM	`<crm-brand-studio>`, `<crm-brand>`, `<crm-sync>` (event bus)	`/embed/crm-elements.js` ✓	`defer`
PIM	`<pim-*>` + `crmPim.resolve()` GID⇄Webflow⇄Xano	`/embed/pim-elements.js` ✓	`defer`
Design	brand theme / tokens (Layer 2 look)	`/brand/<slug>/theme.css` ✓	helmet
Docs modal	`window.crmDocsModal` (docs utility)	Pages `docs-nav.js` (data-modal-only)	`defer`

Bundles are **served from the worker**, so an update ships with `wrangler deploy` — the storefront never re-pastes code. The Stack loader is `window.crmStack` (`data-stack="uikit"`, `data-worker`, `data-shop`; GA4 pulled from `/stack/config` per-shop, MP secret server-side). The Brand Designer loads the SAME loader with `data-stack="uikit,petite-vue,gsap"`.

Placement rule: the **Nav loads in the helmet** (head — no FOUC, paints first). The **Footer loads deferred** (`defer`) at end of body and carries the Shopify / CRM Web Components for **Login + Cart**.

2. ID registry (the `crm-` namespace)

Header nav (`shopify-uikit-nav.liquid`)

ELEMENT	ID	CLASS	NOTES
Nav wrapper / mount	—	`.crm-nav`	petite-vue mount root
Search	`#crm-nav-search`	`.crm-nav-search`	UIkit `search` icon → `routes.search_url`
Cart	`#crm-nav-cart`	`.crm-nav-cart`	UIkit `cart` icon + `[data-cart-count]` badge
Login	`#crm-nav-login`	`.crm-nav-login`	→ `routes.account_login_url`
CTA	—	`.crm-nav-cta`	primary button
Offcanvas (mobile)	`#crm-nav-offcanvas`	—	UIkit offcanvas
Mobile search / cart / login	`#crm-nav-search-m` · `#crm-nav-cart-m` · `#crm-nav-login-m`		inside offcanvas

Footer (`shopify-uikit-footer.liquid` ⏳)

ELEMENT	ID	NOTES
Footer wrapper	`#crm-footer`	render root
Login (web component)	`#crm-nav-login`	shared login id — `<crm-login>` / Shopify Web Component
Cart (web component)	`#crm-nav-cart`	shared cart id — `<crm-cart>`
Footer nav mount	`#crm-footer-nav`	from `GET /nav?menu=footer`

> Cart and Login share **one id each** across nav + footer so a single cart/login component instance binds regardless of which surface triggered it.

Collections (Webflow → sync → worker)

COLLECTION	LIST ELEMENT ID	ITEM / LINK CLASS	MENU KEY
Nav Menu (header)	`#crm-nav-collection`	`.crm-nav-item` / `.crm-nav-link`	`main`
Footer Menu	`#crm-footer-collection`	`.crm-footer-item` / `.crm-footer-link`	`footer`
Tags → Category	`#crm-category-collection`	`.crm-category-item`	(category tables)

Collection field slugs (what the sync reads): `title`, `url`, `active`, `locale` (+ optional `order`, `group`).

---

3. Data endpoints (worker)

ENDPOINT	METHOD	AUTH	PURPOSE
`/stack/config?shop=`	GET	public	GA4 measurement ID (public subset only) ✓
`/nav?shop=&menu=&locale=`	GET	public	named menu (`main` / `footer`), localized ✓
`/nav?shop=&menu=&locale=`	POST	admin/tenant	write a menu (Webflow sync / config app) ✓
`/flow/campaign`	POST	admin/tenant	Shopify Flow action → GA4 Smart Bidding (consent-gated, revenue-weighted) ✓
`/categories?shop=&locale=&kind=`	GET/POST	public / admin	Tags as Category Collection ✓ (kind filter; mirrors /nav)

`menu` resolution: `nav_menus[menu].i18n[locale]` → `[lang]` → `.items`. Each locale is a separately-editable instance (English + globalized variants).

---

4. Tags = Category Collection Tables

Tags are modeled as a **Category Collection** — the same collection→sync→worker pattern as nav, backed by the category tables (Xano `channels(201)` / `channel_membership(202)` / `category_pivot(203)`). A Webflow "Category" collection (`#crm-category-collection`) syncs to `/categories`, and category-driven UI (filters, tag chips, audience membership) reads from it — one authoring surface, localized, projected to every surface.

---

5. Where each piece goes (paste map)

PIECE	THEME LOCATION	LOADING
Stack loader + UIkit CSS + nav `<style>`	`<head>` (helmet)	blocking CSS, `async` JS
Nav markup	top of body / section	server-rendered Liquid
Footer markup + CRM/PIM elements	before `</body>`	`defer`
Brand theme.css (Design)	`<head>`	blocking

Long-term: the worker's Theme App Extension **app embed** injects the helmet + deferred bundles, so there's no manual theme editing.

---

6. Triggers & Actions → SSR Functions

Four axes are modeled as **Triggers** whose **Actions** execute in **server-side functions** (worker functions / Shopify Functions) — never in theme JS. They ride the consent-aware event bus (Tier A), are **fail-closed**, and are audit-logged.

AXIS	TRIGGER (FIRES ON…)	ACTION (SERVER-SIDE)	SSR FUNCTION / SEAM
Brand	brand/theme selected or published	re-theme — emit `theme.css` / tokens for the surface	`/brand/<slug>/theme.css` · Design bundle
QA / PROD	promote between realms (Stage → Prod → Deploy-Live)	gate + swap env-scoped config/creds; fail-closed if connections/approvals incomplete	`/brand/<slug>/promote` · env-label config
Persona	role / entitlement change (Designer, QA, Release Eng)	cap check at the **data plane**; hide-by-cap in UI, enforce in worker	entitlements(190) · `hasCap()` / `userIsDesignerForBrand()`
Event	consent change · cart · AP2 mandate · A2A delegation	run the handler (project to GA4/ESB, gate the cart, settle)	consent-gated `dataLayer` bus → worker/Shopify Functions

The rule: a Trigger is a condition on the bus; its Action is a **deterministic server-side function** (input-bounded, side-effect-scoped), re-checked per loop. Presentation (nav/footer/components) only *reflects* the result — it never decides. This is why the same trigger holds whether a human or an agent fired it.

Backs onto: release personas / separation-of-duties, the brand env realms (Stage-Agency / Prod-Agency / Deploy-Live), and `entitlement_changes(193)` / AP2 `agentic_checkout` on the event bus.

7. Identity spine — every Shopify GID ⇄ same-name Webflow item

Invariant: every Shopify **GID** (`gid://shopify/Product/…`, Collection, Customer, Order, …) has a **same-name item in Webflow**, joined to a **Xano row** as the durable source of truth. One entity, three representations, one natural key.

```
SHOPIFY gid://…  ⇄  XANO row (SoR, natural key + gid)  ⇄  WEBFLOW item (same name/slug)
                          ▲            sockets (CF Worker)          ▲
                          └──────── AI / CF / Xano orchestration ─────┘
```

Rules (from the ORM discipline in `PROCESS-MANAGEMENT-DATA-LAYER.md`):

- **Stable natural key** (SKU/handle/name), not a platform id, is the join — the item survives being re-created in any one platform. The GID is carried as an attribute, not the key.
- **Xano is the source of truth**; Shopify and Webflow are projections. Data flows **out of** Xano; no channel is the only place a fact exists.
- **One writer per field** (price → Xano, published-state → Webflow, fulfillment → Shopify); versioned upserts, idempotent on `(natural_key, source)`.
- **Sockets = CF Worker** owns the mapping + idempotency + conflict resolution; **AI** orchestrates match/normalize/enrich; **Xano** persists. Same seam the nav / footer / category collections ride — the collection items are just GID-keyed rows projected to each surface.

This makes every entity **agent-addressable and consent-qualified** end to end: an AI agent resolves a Shopify GID → Xano row → Webflow item (and back) through one worker socket, under caps + consent, per loop.

8. Semantic wrapper — machine legibility (AEO + a11y)

The **UIkit / semantic wrapper** is what makes every `#crm-` id machine-legible. Because `uk-*` is BEM-namespaced (conflict-free by construction) it supplies the semantic skeleton that utility CSS can't; on top of it we attach a **role + attribute pattern keyed to the id** so screen readers *and* answer engines parse the same structure.

Pattern per addressable element:

LAYER	CARRIES	EXAMPLE (`#CRM-NAV-CART`)
id	the address	`id="crm-nav-cart"`
ARIA role / label	a11y semantics (WCAG)	`aria-label="Cart"`
`data-crm-role`	machine role for AEO / agents	`data-crm-role="cart"`
`data-crm-region`	landmark on the wrapper	`<nav … data-crm-region="primary-nav">`
UIkit BEM class	component structure	`.uk-navbar-item`

Applied on the nav today: `data-crm-region="primary-nav"` on the `<nav>` landmark; `data-crm-role="search|cart|login"` on the controls (+ native `role` / `aria-label`). Same pattern extends to footer, category chips, and every `<crm-*>` component — one wrapper, three readers (browser, screen reader, answer engine / agent).

Scores against the a11y (WCAG) + machine-index (AEO) harness — the wrapper is how we keep both high without hand-tuning each surface.